

Section 3: Generative AI - Fundamentals

Complete Notes with Enterprise Context & Code Snippets

📌 **Course:** Generative AI Application Design and Development 📁 **Section:** 3 | **Videos:** 10-25 | **Duration:** 2hr 2min

How to use this document: Read top to bottom for the first time. Each section builds on the previous one. Enterprise callouts (🏢) highlight how each concept applies in real production systems. Code snippets (💻) show practical implementation where relevant.

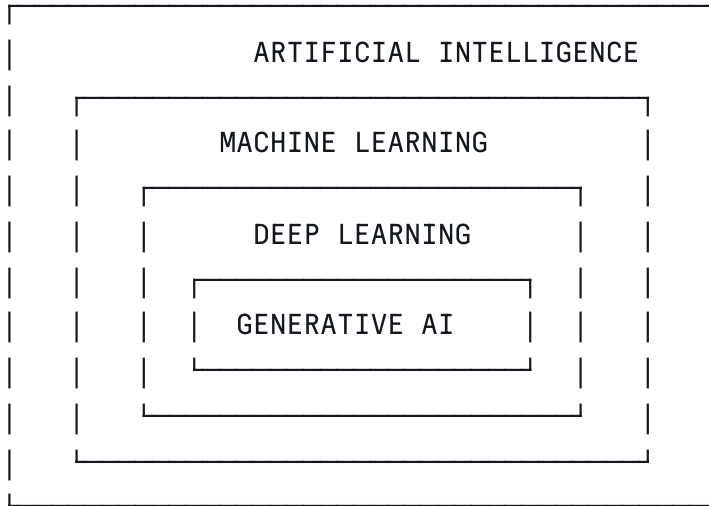
Table of Contents

1. The AI Hierarchy - AI → ML → Deep Learning → GenAI
 2. Neurons, Neural Networks & Deep Learning
 3. Exercise Insight - Neural Network for Math Equations
 4. GenAI Model as a Black Box
 5. Overview of Generative AI Applications
 6. Google Gemini - Accessing Models via API
 7. Introduction to Hugging Face
 8. The Hugging Face Portal
 9. Hugging Face Community & Ecosystem
 10. Intro to NLP - NLU & NLG
 11. NLP with LLMs
 12. Exercise - NLP Tasks with Hugging Face Models
 13. Key Takeaways & The Big Picture
-

1. The AI Hierarchy

The Stack: AI → ML → Deep Learning → GenAI

Think of these as **nested circles**, each being a more specific subset of the one before it.



Artificial Intelligence (AI)

Definition: The broadest category. Any technique that enables machines to mimic human intelligence.

Includes:

- Rule-based systems (if-else logic trees)
- Search algorithms
- Expert systems
- Machine learning (and everything below it)

Enterprise Example 🏢

A bank's fraud detection system in the 1990s used hand-written rules: *"If transaction > \$10,000 AND country != home_country → flag as suspicious"* This is AI, but NOT machine learning. A human wrote every rule manually.

Machine Learning (ML)

Definition: A subset of AI where the system **learns patterns from data** instead of being explicitly programmed with rules.

Core idea:

Traditional Programming: Rules + Data → Output
Machine Learning: Data + Output → Rules (learned automatically)

Types of ML:

Type	What it does	Example
Supervised Learning	Learns from labeled data	Credit scoring
Unsupervised Learning	Finds patterns in unlabeled data	Customer segmentation
Reinforcement Learning	Learns by trial and reward	Game playing, robotics

Enterprise Example

A credit card company trains an ML model on millions of past transactions (labeled "fraud" or "not fraud"). The model learns patterns like: unusual location + high amount + odd time = likely fraud. No human wrote these rules — the model discovered them from data.

Deep Learning (DL)

Definition: A subset of ML that uses **neural networks with many layers** (hence "deep") to learn complex, hierarchical patterns.

Why "deep"?

- Shallow ML: 1-2 layers, learns simple patterns
- Deep Learning: 5, 10, 100+ layers, learns abstract patterns

What deep learning unlocked:

Before DL	After DL
Images needed hand-crafted features	Model learns features automatically
Speech recognition was rigid	Near-human speech recognition
Text translation was rule-based	Fluid, contextual translation

Requires:

- Large volumes of data (millions of examples)
- Significant compute (GPUs/TPUs)
- Long training time (hours to weeks)

Enterprise Example

A healthcare company trains a deep learning model on 2 million chest X-rays to detect pneumonia. The model learns to identify subtle pixel patterns across many layers — edges → shapes → textures → anatomical structures → disease indicators. A radiologist review is still required, but the model flags high-priority cases first.

Generative AI (GenAI)

Definition: A branch of deep learning where models **generate new content** — text, images, audio, video, code — rather than just classifying or predicting.

The fundamental shift:

Traditional ML/DL	Generative AI
"Is this email spam?" (classify)	"Write me a professional email" (generate)
"What is this image of?" (recognize)	"Create an image of a sunset" (generate)
"Will this customer churn?" (predict)	"Summarize why customers are leaving" (generate)

What made modern GenAI possible:

1. **Transformer architecture** (2017, Google) — revolutionary way to process sequences
2. **Massive datasets** — trained on essentially the entire internet
3. **Scale** — billions of parameters, thousands of GPUs
4. **RLHF** — Reinforcement Learning from Human Feedback (makes models helpful and safe)

Enterprise Example

An insurance company builds a GenAI system that:

- Reads a claim document (input)
- Generates a structured summary (output: GenAI)
- Classifies claim priority (output: traditional ML)
- Detects if the claim looks fraudulent (output: deep learning)

A real production system often combines all three types.

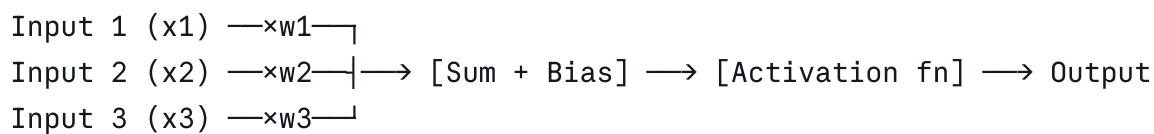
2. Neurons, Neural Networks & Deep Learning

The Biological Inspiration

The artificial neuron is loosely inspired by the human brain's neuron:

Biological Neuron:		Artificial Neuron:	
Dendrites (inputs)	→	Input values (x1, x2, x3)	
Cell body (process)	→	Weighted sum + activation	
Axon (output)	→	Output value	

Anatomy of an Artificial Neuron



Components:

- **Inputs ($\mathbf{x}$):** The data coming in (e.g., pixel values, word embeddings, numbers)
- **Weights ($\mathbf{w}$):** How important each input is — these are **LEARNED** during training
- **Bias ($\mathbf{b}$):** A constant that shifts the output — also learned
- **Activation Function:** Decides whether the neuron "fires" and adds non-linearity

The math (simple form):

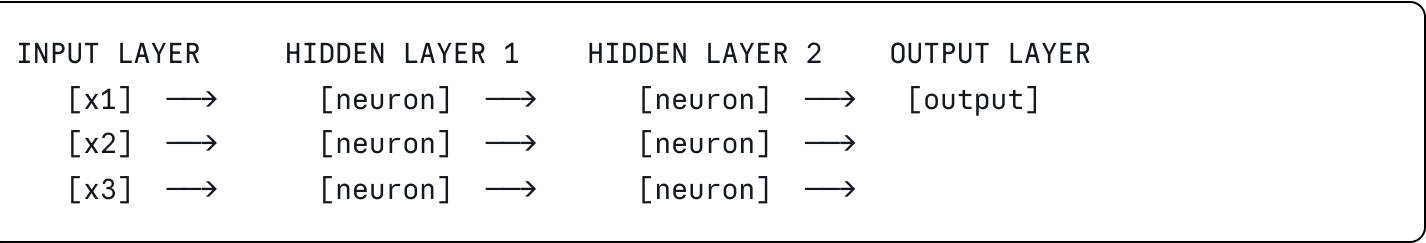
```
output = activation_function( (x1*w1) + (x2*w2) + (x3*w3) + bias )
```

Why Activation Functions Matter

Without activation functions, stacking layers is useless — multiple linear layers collapse into one linear layer. Activation functions introduce **non-linearity**, allowing the network to learn complex patterns.

Activation Function	Formula	When used
ReLU	$\max(0, x)$	Hidden layers (most common)
Sigmoid	$1/(1+e^{-x})$	Binary classification output
Softmax	Normalizes to probabilities	Multi-class classification output
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	Some hidden layers

Layers in a Neural Network



Layer	Role
Input Layer	Receives raw data (pixels, numbers, word tokens)
Hidden Layers	Learn increasingly abstract representations
Output Layer	Produces the final prediction or value

What each hidden layer learns (example for image recognition):

- Layer 1: Edges and simple lines
- Layer 2: Shapes and corners
- Layer 3: Object parts (eyes, wheels, letters)
- Layer 4+: Full objects, concepts

Training a Neural Network

Training is the process of **adjusting weights** so the network gets better at its task.

TRAINING LOOP

1. Forward Pass: Data flows through network → prediction
2. Loss Calculation: How wrong is the prediction?
3. Backward Pass (Backpropagation): Calculate gradients
4. Weight Update (Optimizer): Adjust weights slightly
5. Repeat for thousands/millions of examples

Key terms:

- **Loss Function:** Measures how wrong the model is (lower = better)
- **Optimizer:** Algorithm that updates weights (e.g., Adam, SGD)
- **Epoch:** One complete pass through the entire training dataset
- **Learning Rate:** How big each weight adjustment step is

Enterprise Example 🏢

Training a document classification model for a law firm:

- Input: 500,000 legal documents
- Labels: "Contract", "Invoice", "Court Filing", "Correspondence"
- Training: ~50 epochs, loss goes from 2.3 → 0.08
- Result: 94% accuracy on new documents
- Cost: ~\$800 on cloud GPUs for 6 hours of training

Types of Deep Learning Networks

Network Type	Architecture	Best For	Enterprise Use Case
Feedforward (FNN)	Layers in sequence, no loops	Regression, classification	Price prediction, risk scoring
Convolutional (CNN)	Spatial filters + pooling	Images, video	Defect detection in manufacturing
Recurrent (RNN)	Has memory of previous inputs	Sequences, time series	Stock price prediction

Network Type	Architecture	Best For	Enterprise Use Case
LSTM	Advanced RNN with long memory	Long sequences, translation	Document summarization
Transformer	Attention mechanism	Text, multimodal	All modern LLMs (GPT, Claude, Gemini)

Important for GenAI developers: You don't need to implement these from scratch. But understanding *which architecture does what* helps you choose the right model for your use case and explain model behavior to stakeholders.

Parameters – The Size of a Model

Parameters are all the weights and biases in a neural network. They represent what the model has "learned."

Model	Parameters	Notes
Simple regression model	~100	Toy example
ResNet-50 (image model)	~25 million	Production image classification
GPT-2	~1.5 billion	Early LLM
GPT-3	~175 billion	Breakthrough LLM
Llama 3 70B	~70 billion	Modern open-source LLM
GPT-4 (estimated)	~1.7 trillion	State of the art

Enterprise implication 🏢:

- More parameters = more capable BUT:
- More expensive to run (inference cost)
 - Slower response time (latency)
 - Requires more memory (VRAM)
- In production, you pick the **smallest model that meets your accuracy requirements** — not always the biggest.

3. Exercise Insight - Neural Network for Math Equations

What this exercise demonstrates

The exercise has you build a simple neural network to learn a mathematical relationship (e.g., $y = 3x + 5$). This is intentionally simple, but reveals a critical production truth.

The Data Distribution Problem

```
python

# Conceptual example of what the exercise shows

# Training data: x values between 0 and 100
x_train = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
y_train = [5, 35, 65, 95, 125, 155, 185, 215, 245, 275, 305]

# Model trains, learns the pattern y = 3x + 5

# Works well WITHIN the training range:
model.predict(50) # → ~155 ✓ Accurate

# Degrades OUTSIDE the training range:
model.predict(500) # → ??? ✗ Unreliable (extrapolation)
model.predict(-50) # → ??? ✗ Unreliable (never seen negative x)
```

Why this matters enormously for LLMs

This simple exercise reveals the **most important concept** for GenAI production systems:

▮ **Models perform best on data similar to what they were trained on.**

Applied to LLMs:

- If an LLM was trained mostly on English text → it performs worse on rare languages
- If asked about events after its training cutoff → it hallucinates
- If given a domain it rarely saw (e.g., niche legal jargon) → responses are less reliable

Enterprise Example 🏢

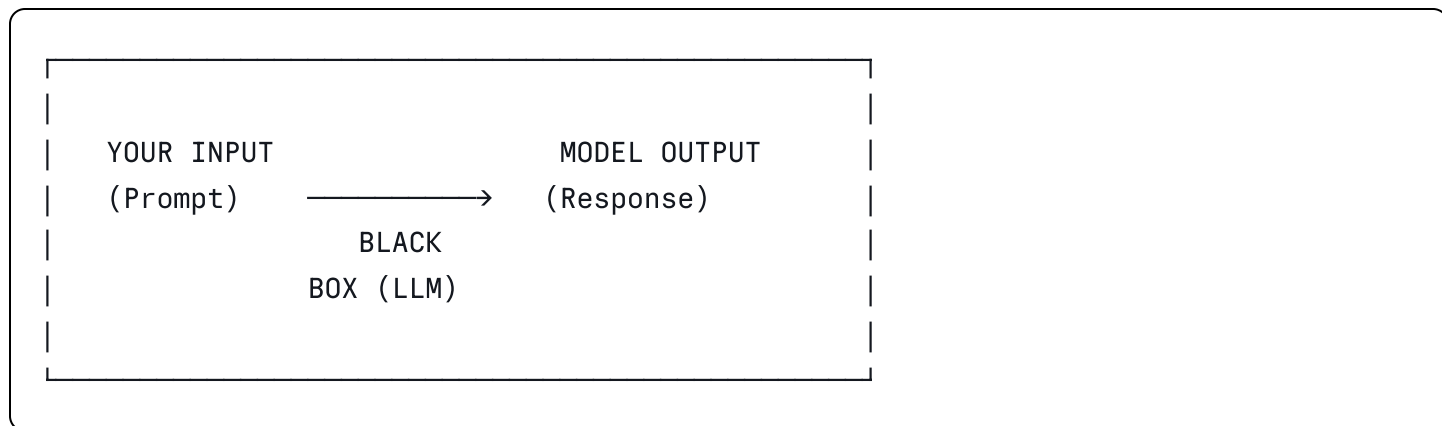
A healthcare company deploys an LLM to answer patient questions. The model was trained on general medical text but NOT on the hospital's specific drug formulary or internal protocols. When patients ask about specific medications the hospital stocks, the model gives generic answers or hallucinates drug names.

Solution: RAG (Retrieval-Augmented Generation) — feed the model relevant internal documents at query time. This is why RAG exists.

4. GenAI Model as a Black Box

The Black Box Mental Model

As a GenAI application developer, you interact with LLMs primarily as a **black box**:



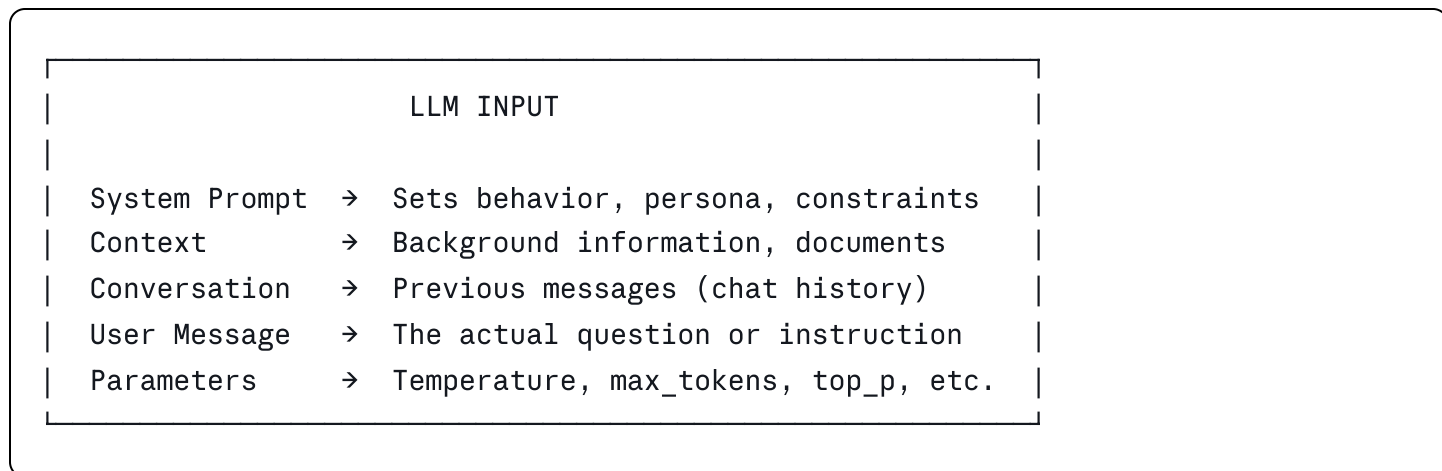
You control:

- What goes **in** (prompt, system prompt, context, parameters)
- What you do with what comes **out** (parse, validate, route, display)

You do NOT control:

- The model's internal weights
- How it generates tokens
- Whether it's deterministic (it's not — by default)

What Goes Into the Black Box



Inference Parameters (The Knobs You Control)

Parameter	What it controls	Low value	High value
Temperature	Randomness/creativity	Deterministic, focused	Creative, unpredictable
Max Tokens	Length of response	Short responses	Longer responses
Top-p	Probability sampling range	Conservative	More varied
Top-k	How many tokens to consider	Focused	More diverse

Enterprise use cases by temperature:

Use Case	Temperature	Why
Code generation	0.0 - 0.2	Exact, reproducible output
Data extraction	0.0	Deterministic parsing
Customer support	0.3 - 0.5	Consistent but natural
Creative writing	0.7 - 1.0	Varied, creative responses
Brainstorming	0.8 - 1.2	Maximum diversity



Code snippet:

```
python
```

```

import os
from groq import Groq

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant. Be concise."
        },
        {
            "role": "user",
            "content": "Explain how a transformer works."
        }
    ],
    temperature=0.3,      # Low = focused, consistent
    max_tokens=500,       # Limit response length
    top_p=0.9
)

print(response.choices[0].message.content)
print(f"Tokens used: {response.usage.total_tokens}")

```

What Comes Out of the Black Box

The model outputs **tokens** (not characters, not words — tokens). Tokens are generated **one at a time**, left to right.

```

Prompt: "The capital of France is"
Token 1: " Paris"
Token 2: "."
Output: "The capital of France is Paris."

```

This is why LLMs can be slow for long outputs — they generate sequentially, not all at once.

LLMs are Probabilistic, Not Deterministic

```

Question: "What is 2 + 2?"
Run 1 (temperature=0): "4"
Run 1 (temperature=0): "4"    ← Same every time at temp=0

```

Question: "Write me a poem about clouds"

Run 1 (temperature=0.8): "Fluffy giants drift above..."

Run 2 (temperature=0.8): "Silent travelers of the sky..." ← Different every time

Enterprise implication 🏢:

For any business process that requires **auditability** (finance, legal, healthcare), you must:

1. Log every prompt AND response
2. Use low temperature for consistency
3. Validate outputs before using them in decisions
4. Never treat LLM output as ground truth — treat it as a draft

5. Overview of Generative AI Applications

The GenAI Application Landscape

GenAI isn't one thing — it's a family of capabilities. Here's the full map:

GENERATIVE AI APPLICATIONS			
TEXT Generation	IMAGE Generation	AUDIO Generation	CODE Generation
Summarize	Create art	Text-to-Speech	Write code
Translate	Edit photos	Speech-to-Text	Review code
Q&A	Generate from text	Speech-to-Speech	Debug code
Chat	Style transfer	Text-to-Music	Generate SQL
Extract		Music gen	Write tests
Classify		Voice clone	

Enterprise GenAI Application Patterns

These are the patterns you will build throughout this course:

Pattern 1: Q&A / Chatbot

User Question → LLM → Answer

Example: Internal HR bot that answers employee policy questions

Pattern 2: Summarization

Long Document → LLM → Concise Summary

Example: Legal contract summarizer that highlights key clauses and risks

Pattern 3: Extraction

Unstructured Text → LLM → Structured JSON

Example: Extract invoice fields (vendor, amount, date, line items) from PDFs

Pattern 4: Classification

Input Text → LLM → Category Label

Example: Route customer support tickets to the right department automatically

Pattern 5: RAG (Retrieval-Augmented Generation)

Question → Search your docs → Relevant chunks + Question → LLM → Grounded Answer

Example: Customer support bot that answers only from your product documentation

Pattern 6: Agentic / Tool Use

Goal → LLM decides steps → Calls tools (APIs, DBs, search) → LLM synthesizes → Answer

Example: Sales agent that checks CRM, sends emails, and schedules meetings autonomously

Enterprise Example — Full System

A large insurance company builds a claims processing system:

- **Extraction:** Pull structured data from claim forms (PDF → JSON)
- **Classification:** Categorize claim type (auto, home, health)
- **Q&A:** Answer adjuster questions about specific claims
- **Summarization:** Generate claim summaries for management reports
- **Agentic:** Automatically trigger workflows (send to assessor, request documents)

This is a single product, but it uses 5 different GenAI patterns.

6. Google Gemini - Accessing Models via API

Why Multiple Providers Matter

The course uses both Groq and Google Gemini. This is intentional — in production, you rarely lock into a single provider.

Google Gemini Model Family

Model	Speed	Capability	Best For
Gemini 1.5 Flash	Very fast	Good	High-volume, cost-sensitive apps
Gemini 1.5 Pro	Moderate	Excellent	Complex reasoning, long context
Gemini Ultra	Slow	Best	Most demanding tasks

Gemini's key differentiator: Extremely large context window (up to 1 million tokens in 1.5 Pro). This means you can feed entire books, large codebases, or hours of video transcripts.

Accessing Gemini via Python

 **Code snippet:**

```
python
```

```
import os
import google.generativeai as genai

# Configure with API key
genai.configure(api_key=os.environ.get("GOOGLE_API_KEY"))

# Initialize the model
model = genai.GenerativeModel('gemini-1.5-flash')

# Simple text generation
response = model.generate_content(
    "Explain transformer architecture in simple terms."
)
print(response.text)

# With system instructions and chat history
model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    system_instruction="You are an expert data scientist. Be precise and technical."
)

chat = model.start_chat(history=[])
response = chat.send_message("What is attention mechanism in transformers?")
print(response.text)
```

Provider Comparison for Enterprise

Criteria	OpenAI (GPT-4)	Google (Gemini)	Anthropic (Claude)	Groq (Llama)
Capability	Excellent	Excellent	Excellent	Good
Speed	Moderate	Fast	Moderate	Very Fast
Context Window	128K	1M	200K	8K-32K
Cost	High	Moderate	Moderate	Low/Free
Data Privacy	Standard	Standard	Strong	Standard
Free Tier	No	Yes	No	Yes

Enterprise Decision Framework 🏢:

Choose provider based on:

- **Latency requirements** → Groq wins for speed
- **Context size needs** → Gemini 1.5 Pro for huge documents
- **Safety/compliance** → Anthropic Claude for regulated industries
- **Multimodal needs** → GPT-4o or Gemini for images + text
- **Cost at scale** → Run the math: (requests/day × avg_tokens × price_per_token)

7. Introduction to Hugging Face

What is Hugging Face?

Hugging Face is the **GitHub of AI** — a platform and company that:

1. Hosts thousands of open-source AI **models**
2. Hosts curated **datasets** for training
3. Provides **Spaces** — live demos and hosted apps
4. Provides the `transformers` Python library — the most widely used AI library

HUGGING FACE		
MODELS	DATASETS	SPACES
70,000+ pre-trained models	15,000+ curated datasets	Live demos of models Built with Gradio or Streamlit
Free to download	Free to download	Free to host and use

Why Hugging Face Matters for Enterprise

Option A: Use a hosted API (OpenAI, Groq, Gemini)

-  Easy to get started
-  No infrastructure
-  Your data goes to a third party

- ❌ Ongoing API costs at scale
- ❌ No customization of model weights

Option B: Use Hugging Face models (self-hosted)

- ✅ Full data privacy — runs in your own infrastructure
- ✅ One-time cost (compute), no per-token pricing
- ✅ Can fine-tune on your own data
- ❌ Requires ML engineering to deploy and maintain
- ❌ Your team manages the infrastructure

Enterprise Example 🏢

A bank cannot send customer financial data to OpenAI's servers due to data residency laws. They use Hugging Face to download a Llama 3 model, run it on their own AWS VPC (private cloud), and build their internal chatbot — zero data leaves their infrastructure.

The `transformers` Library

The backbone of Hugging Face's ecosystem:

📄 Code snippet — the simplest possible usage:

```
python

from transformers import pipeline

# One line to load a model and run inference
classifier = pipeline("sentiment-analysis")

result = classifier("I absolutely love this product!")
print(result)
# → [{'label': 'POSITIVE', 'score': 0.9998}]

# Works for many tasks:
summarizer = pipeline("summarization")
translator = pipeline("translation_en_to_fr")
generator = pipeline("text-generation", model="gpt2")
qa_model = pipeline("question-answering")
```

The `pipeline` abstraction hides everything — model download, tokenization, inference, post-processing. It's perfect for prototyping.

Model Cards - The Passport of Every Model

Every model on Hugging Face has a **Model Card** — a structured document that tells you:

- What the model does
- What data it was trained on
- Its limitations and biases
- How to use it (code examples)
- License (can you use it commercially?)

Enterprise implication 🏢:

Before using any Hugging Face model in production, ALWAYS read the Model Card:

- **License:** Apache 2.0 (free commercial use) vs CC-BY-NC (non-commercial only)
- **Biases:** What populations or languages is it worse at?
- **Intended use:** What was it designed for?
- **Out-of-scope use:** What should it NOT be used for?

Using a model with a license that doesn't permit commercial use in your product is a **legal risk**.

8. The Hugging Face Portal

Key Sections of the Portal

Models Hub

- Search by task: text-classification, summarization, translation, etc.
- Filter by language, license, size
- Each model shows: downloads per month, last updated, license

Datasets Hub

- Curated datasets for training and evaluation
- Includes: SQUAD (QA), GLUE (NLP benchmarks), CommonCrawl (web text), etc.

Spaces

- Live, interactive demos
- Anyone can deploy a model demo here using Gradio or Streamlit

- Free GPU available for public demos

Finding the Right Model

- Step 1: Go to huggingface.co/models

Step 2: Filter by TASK (e.g., "text-classification")

Step 3: Filter by LICENSE (for commercial use: Apache 2.0, MIT)

Step 4: Sort by DOWNLOADS (popular = battle-tested)

Step 5: Read the MODEL CARD

Step 6: Test in the inference widget (right on the page)

Step 7: Run locally and evaluate on YOUR data

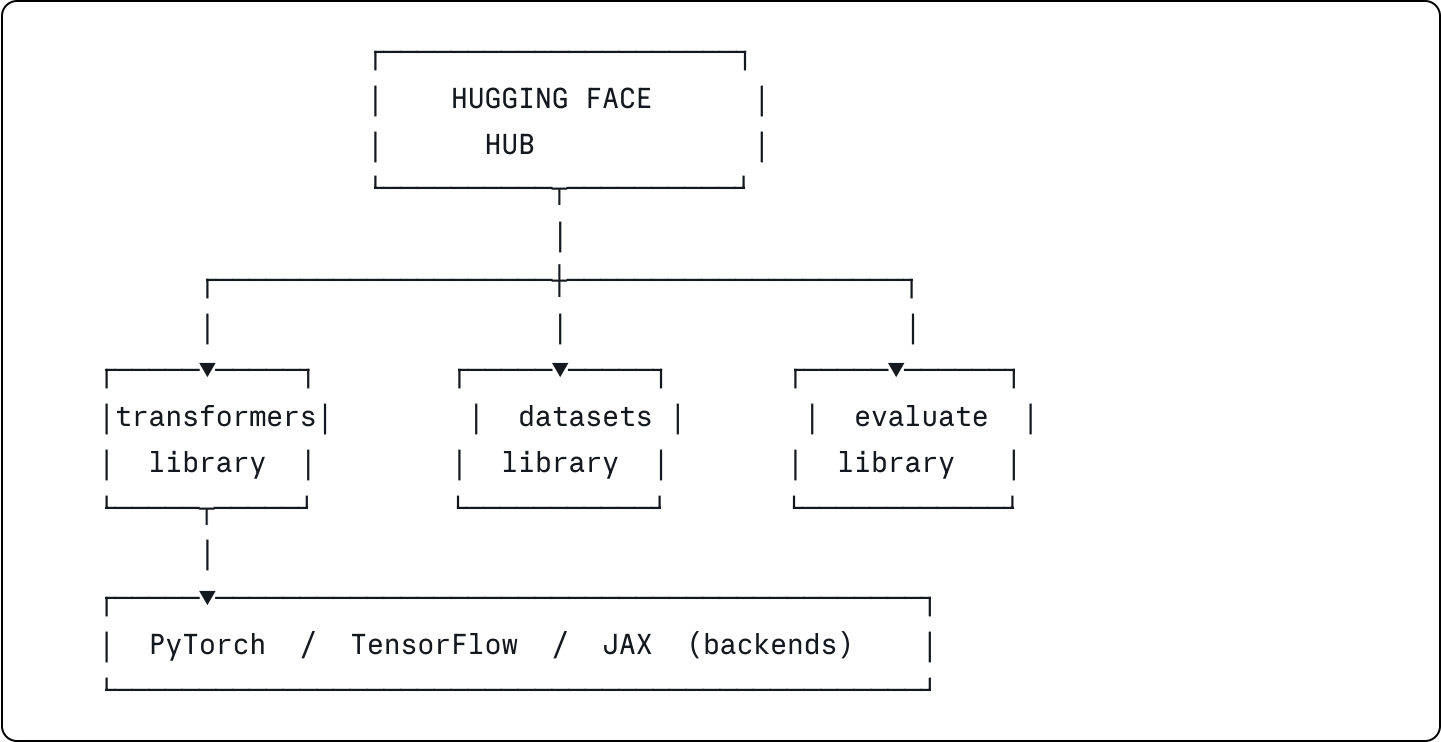
Most Commonly Used Models by Task

Task	Popular Models
Text Generation	<code>meta-llama/Llama-3-8b</code> , <code>mistralai/Mistral-7B</code>
Embeddings	<code>sentence-transformers/all-MiniLM-L6-v2</code>
Sentiment Analysis	<code>distilbert-base-uncased-finetuned-sst-2-english</code>
Summarization	<code>facebook/bart-large-cnn</code>
Translation	<code>Helsinki-NLP/opus-mt-en-fr</code>
Question Answering	<code>deepset/roberta-base-squad2</code>
Named Entity Recognition	<code>dslim/bert-base-NER</code>

9. Hugging Face Community & Ecosystem

The Open Source AI Ecosystem

Hugging Face is the center of the open-source AI world:



Key Hugging Face Libraries

Library	Purpose	When to use
transformers	Load and run models	Always — it's the main one
datasets	Load training/evaluation data	When fine-tuning or evaluating
evaluate	Compute metrics (accuracy, F1, BLEU)	When evaluating model quality
peft	Parameter-efficient fine-tuning (LoRA)	When fine-tuning large models cheaply
accelerate	Distribute training across GPUs	When training at scale
diffusers	Image generation (Stable Diffusion)	For image GenAI pipelines

The Community Advantage

- 70,000+ models contributed by researchers and companies worldwide
- Models from: Google, Meta, Microsoft, Mistral, Stability AI, and thousands of researchers
- Forums, Discord, papers shared openly
- Leaderboards for benchmarking models on standard tasks

Enterprise Example

A retail company needs a sentiment analysis model for product reviews in 5 languages (English, Spanish, French, German, Japanese). Instead of:

- Training from scratch (\$50,000+ in compute)
- Buying an API subscription

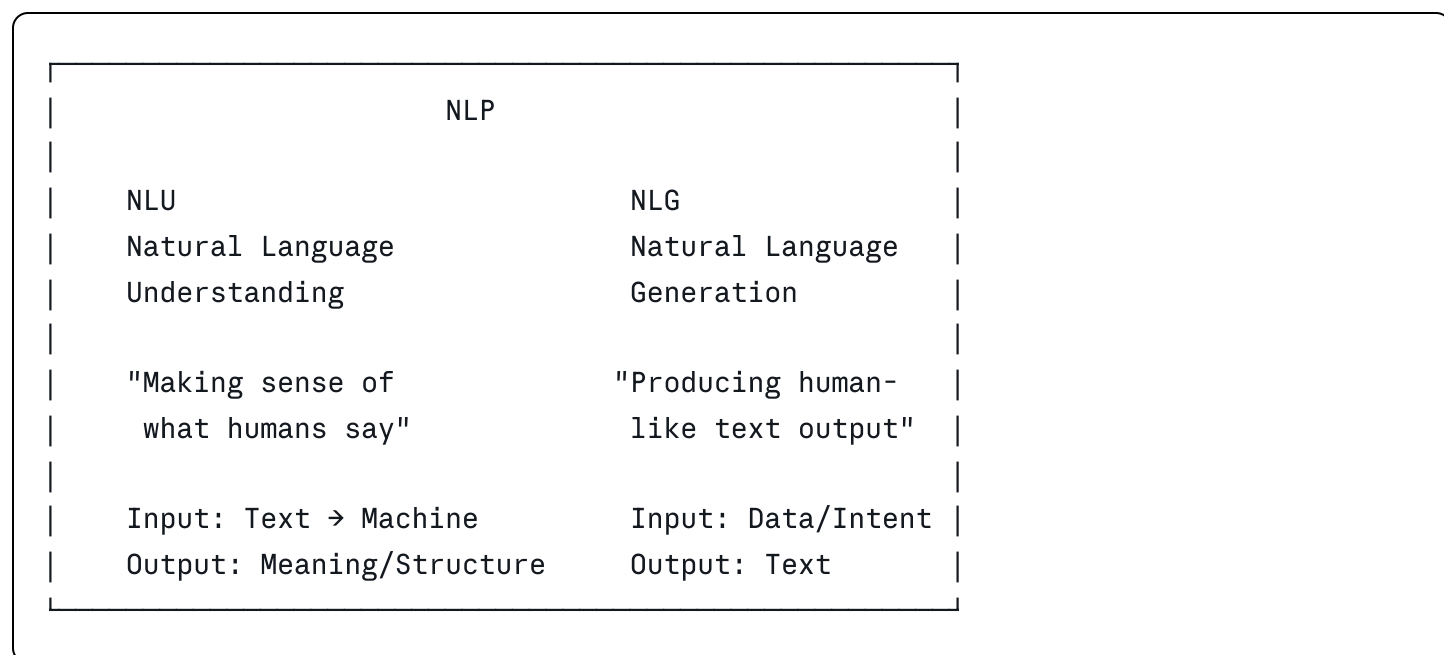
They search Hugging Face, find `nlptown/bert-base-multilingual-uncased-sentiment` — a pre-trained model that handles all 5 languages, fine-tuned on product reviews. They download it, test it on their data, and deploy it in a day.

10. Intro to NLP - NLU & NLG

What is NLP?

Natural Language Processing (NLP) is the field of AI focused on enabling computers to understand, interpret, and generate human language.

NLP has two major sub-fields:



NLU - Natural Language Understanding

NLU is about **comprehending** text — extracting meaning, intent, structure.

Core NLU Tasks:

Task	What it does	Example Input	Example Output
Sentiment Analysis	Detect emotion/opinion	"This product is terrible"	NEGATIVE (0.98)
Named Entity Recognition (NER)	Identify entities in text	"Apple released iPhone in Cupertino"	Apple=ORG, iPhone=PRODUCT, Cupertino=LOC
Intent Classification	Understand what user wants	"Book me a flight to Delhi"	intent: BOOK_FLIGHT
Text Classification	Categorize text	(Email text)	SPAM / NOT_SPAM
Relationship Extraction	Find how entities relate	"Elon Musk founded SpaceX"	Musk → FOUNDED → SpaceX
Coreference Resolution	Resolve pronouns	"John said he was tired"	he = John

NLU Code snippet (NER):

```
python

from transformers import pipeline

# Named Entity Recognition
ner = pipeline("ner", grouped_entities=True)

text = "Satya Nadella, CEO of Microsoft, announced a $10 billion investment in OpenAI"

entities = ner(text)
for entity in entities:
    print(f"{entity['word']}:20} → {entity['entity_group']:10} (confidence: {entity['score']})")

# Output:
# Satya Nadella      → PER      (confidence: 0.99)
# Microsoft          → ORG      (confidence: 0.99)
# $10 billion        → MONEY    (confidence: 0.97)
# OpenAI             → ORG      (confidence: 0.98)
```

Enterprise Example — NER in Document Processing:

A law firm processes thousands of contracts daily. An NER model automatically extracts:

- Party names (PER, ORG)
- Dates (DATE)
- Monetary values (MONEY)
- Locations (LOC)

This structured data populates a contract management database automatically, replacing 40 hours/week of manual data entry.

NLG - Natural Language Generation

NLG is about **producing** human-like text from data, intent, or other text.

Core NLG Tasks:

Task	Input	Output	Enterprise Example
Text Generation	Prompt/context	Continuation	Email drafting assistant
Summarization	Long document	Short summary	Meeting notes summarizer
Translation	Text in language A	Text in language B	Multilingual customer support
Question Answering	Question + context	Answer	Internal knowledge base bot
Data-to-Text	Structured data (JSON/table)	Natural language description	Auto-generate financial reports
Paraphrasing	Original text	Rephrased version	SEO content variation

NLG Code snippet (Summarization):

```
python
```

```

from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

long_text = """
The Board of Directors of Acme Corporation met on March 15th to discuss
quarterly results. Revenue for Q1 2024 reached $2.3 billion, representing
a 12% year-over-year increase. Operating expenses grew by 8%, resulting
in an operating margin improvement of 150 basis points to 18.5%.
The company announced plans to expand into three new markets: Southeast Asia,
Eastern Europe, and Latin America, with a combined investment of $500 million
over the next three years. The Board approved a share buyback program worth
$200 million. Chief Executive Officer Jane Smith expressed confidence in
the company's strategic direction and reiterated full-year guidance of
$9.5 billion in revenue.
"""

summary = summarizer(long_text, max_length=80, min_length=30)
print(summary[0]['summary_text'])

# Output:
# "Acme Corporation reported Q1 revenue of $2.3 billion, up 12% year-over-year.
# The company plans to expand into Southeast Asia, Eastern Europe, and Latin
# America with a $500M investment and approved a $200M share buyback."

```

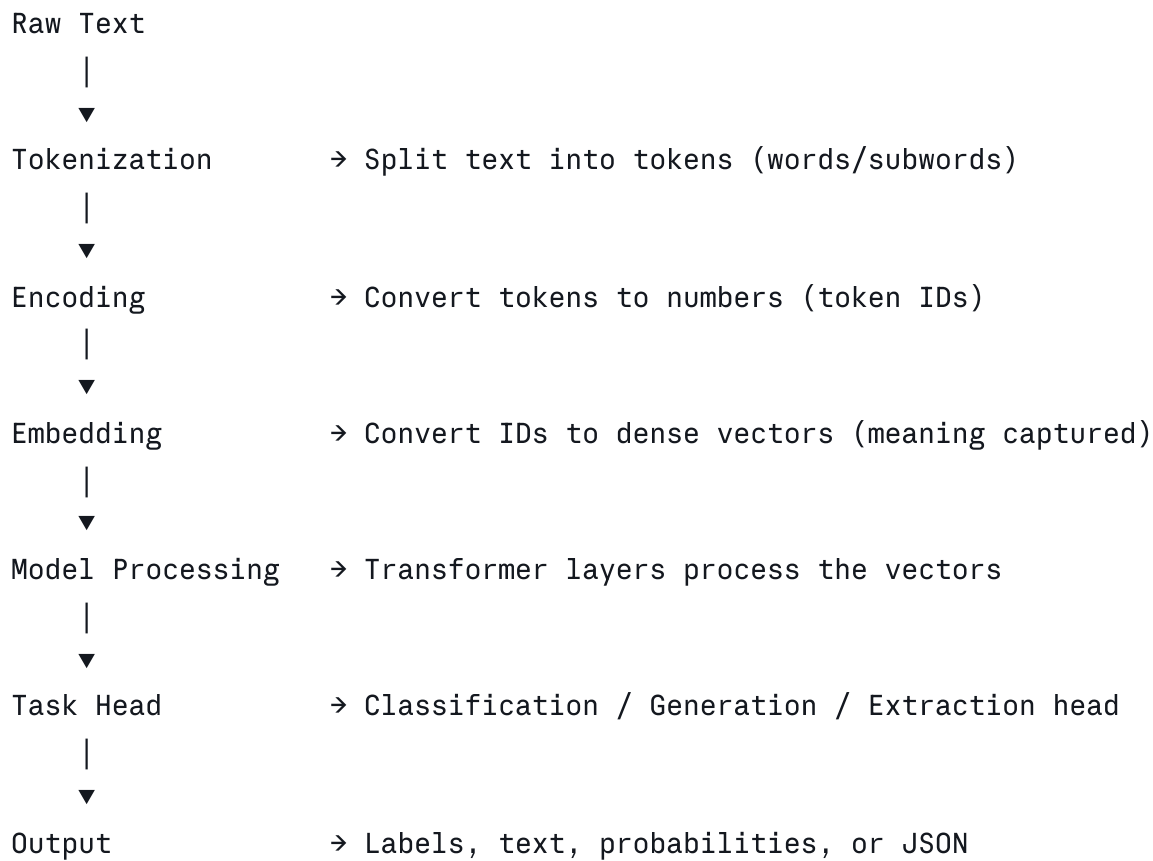
Enterprise Example — Data-to-Text:

A financial services company generates personalized monthly statements for 2 million customers. Instead of templates, an NLG model generates natural language summaries:

Input (JSON): `{spending: 45000, savings_rate: 12%, top_category: "dining", vs_last_month: "+8%"}` Output: *"You spent ₹45,000 this month, 8% more than last month. Your top spending category was dining. Your savings rate of 12% is above the national average — great work!"*

The Full NLP Pipeline

Modern NLP tasks follow a pipeline:



11. NLP with LLMs

How LLMs Changed NLP

Before LLMs, every NLP task needed its **own specialized model**:

- Sentiment model → trained specifically for sentiment
- NER model → trained specifically for NER
- Translation model → trained specifically for translation

With LLMs, **one model does all NLP tasks** via prompting:

Before LLMs:

Sentiment task	→ SentimentBERT model
NER task	→ BERT-NER model
Summarization	→ BART model
Translation	→ MarianMT model
QA task	→ RoBERTa-QA model

With LLMs:

All tasks → GPT-4 / Claude / Llama (via different prompts)

Why LLMs are So Good at NLP

LLMs are trained on a task called **next-token prediction** at massive scale:

Training: Given "The capital of France is ___", predict " Paris"

Training: Given "This movie was absolutely ___", predict " terrible" or " amazing"

Training: Billions of such examples from the internet

By learning to predict the next token across billions of examples, the model implicitly learns:

- Grammar and syntax (NLU)
- Facts about the world (knowledge)
- How to write coherently (NLG)
- Reasoning patterns (inference)

The Transformer Architecture (High-Level)

The architecture behind all modern LLMs:

TRANSFORMER

Input: "How does a car work?"

↓

Tokenize: [How, does, a, car, work, ?]

↓

Embed: Convert to vectors

↓

Self-Attention: Each token attends to
all other tokens (captures context)

↓

Feed-Forward Layers: Process representations

↓

Output: Generate next token probabilities

↓

"A car works by converting fuel into..."

Self-Attention — The Key Innovation:

Self-attention allows the model to understand that in: *"The animal didn't cross the street because it was too tired"* "it" refers to "animal" (not "street") — this contextual understanding is what makes transformers powerful.

LLMs as Zero-Shot, Few-Shot, and Fine-Tuned NLP Tools

Zero-shot: Ask the model to do a task it wasn't explicitly trained on, just by describing it in the prompt.

```
python
```

```
# Zero-shot classification - no training examples provided
prompt = """
Classify this customer review as POSITIVE, NEGATIVE, or NEUTRAL.
Review: "The delivery was late but the product quality is excellent."
Classification:
"""
```

Few-shot: Provide a few examples in the prompt to guide the model.

```
python
```

```
# Few-shot classification – examples in the prompt
prompt = """
Classify customer reviews:

Review: "Works perfectly, fast delivery!" → POSITIVE
Review: "Completely broken, waste of money" → NEGATIVE
Review: "It's okay, nothing special" → NEUTRAL

Review: "The delivery was late but the product quality is excellent." →
"""
```

Fine-tuning: Actually train the model further on your specific data.

- Most expensive approach
- Highest accuracy on your specific domain
- Used when zero/few-shot isn't good enough

Enterprise Decision Framework 🏢:

```
Start with zero-shot
  ↓ (if accuracy insufficient)
Try few-shot (add examples to prompt)
  ↓ (if accuracy still insufficient)
Try prompt engineering (better system prompts, chain-of-thought)
  ↓ (if accuracy still insufficient)
Fine-tune a smaller model on your data
  ↓ (if you need full control)
Train from scratch (extremely rare, very expensive)
```

Key NLP Metrics for Enterprise Evaluation

When evaluating an NLP model for production, these are the metrics you need to know:

Metric	What it measures	Used for
Accuracy	% of correct predictions	Classification
Precision	Of positives predicted, how many are actually positive	When false positives are costly
Recall	Of actual positives, how many did we catch	When false negatives are costly
F1 Score	Harmonic mean of precision and recall	Balanced classification
BLEU Score	Quality of generated text vs reference	Translation, summarization
ROUGE Score	Overlap between generated and reference summary	Summarization
Perplexity	How well model predicts test data	Language modeling

Enterprise Example 🏢:

A medical NLP system flags clinical notes for urgent review. Here, **recall** is more important than precision:

- Missing a genuine urgent case (false negative) = patient harm
- Flagging a non-urgent case (false positive) = extra work, but safe

The team optimizes for $\text{recall} \geq 0.98$, accepts lower precision.

12. Exercise – NLP Tasks with Hugging Face Models

What you're doing in this exercise

You run various NLP tasks using Hugging Face's `pipeline` API. This is where everything in this document becomes hands-on.

The Six Core Tasks to Try

 **Complete exercise code:**

```
python
```

```

from transformers import pipeline

print("=" * 60)
print("1. SENTIMENT ANALYSIS")
print("=" * 60)
sentiment = pipeline("sentiment-analysis")
reviews = [
    "This is the best product I've ever bought!",
    "Terrible experience, will never buy again.",
    "It's fine, does what it's supposed to do."
]
for review in reviews:
    result = sentiment(review)[0]
    print(f"Text: {review[:50]}...")
    print(f"Result: {result['label']} ({result['score']:.2%})\n")

print("=" * 60)
print("2. NAMED ENTITY RECOGNITION")
print("=" * 60)
ner = pipeline("ner", grouped_entities=True)
text = "Tim Cook announced Apple's new product at the Steve Jobs Theater in Cupertino"
entities = ner(text)
for e in entities:
    print(f"{e['word'][:25]} → {e['entity_group']} ({e['score']:.2%})")

print("=" * 60)
print("3. QUESTION ANSWERING")
print("=" * 60)
qa = pipeline("question-answering")
context = """
Hugging Face was founded in 2016 by Clément Delangue, Julien Chaumond,
and Thomas Wolf. The company is headquartered in New York City and Paris.
It provides tools for building, training, and deploying machine learning models.
"""
questions = [
    "When was Hugging Face founded?",
    "Where is Hugging Face headquartered?",
    "Who founded Hugging Face?"
]
for q in questions:
    result = qa(question=q, context=context)

```

```

print(f"Q: {q}")
print(f"A: {result['answer']} (confidence: {result['score']:.2%})\n")

print("=" * 60)
print("4. SUMMARIZATION")
print("=" * 60)
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
article = """
Artificial intelligence is transforming industries across the globe.
From healthcare to finance, companies are deploying AI systems to automate
routine tasks, improve decision-making, and create new products. The technology
has advanced rapidly due to increases in computing power, availability of large
datasets, and breakthroughs in deep learning architectures, particularly
the transformer model introduced in 2017. Major technology companies including
Google, Microsoft, and Amazon have made substantial investments in AI research
and infrastructure, while startups have raised billions in venture capital
to develop specialized AI applications.
"""
summary = summarizer(article, max_length=60, min_length=20)
print(f"Original length: {len(article.split())} words")
print(f"Summary: {summary[0]['summary_text']}")

print("=" * 60)
print("5. TRANSLATION")
print("=" * 60)
translator = pipeline("translation_en_to_fr")
text = "Artificial intelligence is changing the way we work and live."
result = translator(text)
print(f"English: {text}")
print(f"French: {result[0]['translation_text']}")

print("=" * 60)
print("6. TEXT GENERATION")
print("=" * 60)
generator = pipeline("text-generation", model="gpt2")
prompt = "The future of artificial intelligence in enterprise is"
result = generator(prompt, max_length=60, num_return_sequences=1)
print(f"Prompt: {prompt}")
print(f"Generated: {result[0]['generated_text']}")

```

What to Observe

When running these exercises, notice:

- **Speed difference** between small models (distilbert) and large models (bart)
- **Confidence scores** — a low-confidence result is a signal not to trust the output
- **Failure modes** — feed unusual inputs and observe where models break
- **Model size vs quality tradeoff** — smaller models are faster but less accurate

13. Key Takeaways & The Big Picture

The Mental Model for a GenAI Developer

WHAT A GENAI DEVELOPER DOES

1. Understand what LLMs can and cannot do

2. Select the right model for the task

3. Design effective prompts (system + user)

4. Call the model via API or local inference

5. Parse, validate, and use the output

6. Handle errors, rate limits, and costs

7. Monitor, log, and improve over time

YOU ARE THE ARCHITECT OF AI SYSTEMS

The model is a powerful component – not the whole system

Summary Table: All Concepts in One View

Concept	What it is	Why it matters in production
AI → ML → DL → GenAI	Nested hierarchy of techniques	Helps you know what tool to reach for
Neural Network	Layers of mathematical functions	Foundation of all modern AI
Parameters	Learned weights in a model	More = capable but expensive
Transformer	Architecture behind LLMs	Powers every modern LLM
Black Box Model	LLM as input→output function	You control prompts, not weights

Concept	What it is	Why it matters in production
Temperature	Randomness of output	Low for precision, high for creativity
Token	Basic unit of LLM processing	Determines speed and cost
Hugging Face	Open-source AI platform	Find, test, and deploy models
Model Card	Model documentation	Licenses, limitations, biases
NLP	Processing human language with AI	The domain LLMs operate in
NLU	Understanding text	Sentiment, NER, classification
NLG	Generating text	Summarization, translation, chat
Zero-shot	No examples needed	Fastest to try
Few-shot	Examples in prompt	Improves accuracy quickly
Fine-tuning	Train on your data	Highest accuracy, highest cost
RAG	Retrieve + generate	Grounds LLMs in your data

The Learning Path Ahead

Section 3 (Current): Fundamentals 

↓

Prompt Engineering → System prompts, few-shot, chain-of-thought

↓

LLM API Development → Python SDKs, streaming, error handling

↓

RAG Systems → Embeddings, vector databases, retrieval

↓

Agentic AI → LangGraph, tool use, multi-agent systems

↓

Production Patterns → Observability, cost management, deployment

Quick Reference - Commands to Remember

```
bash
```

```
# Install Hugging Face transformers
uv add transformers torch

# Install specific provider SDKs
uv add openai          # OpenAI
uv add anthropic       # Anthropic Claude
uv add groq            # Groq
uv add google-generativeai # Google Gemini

# Set API keys (never hardcode these)
export OPENAI_API_KEY="sk-..."
export GROQ_API_KEY="gsk-..."
export GOOGLE_API_KEY="AIza..."
export ANTHROPIC_API_KEY="sk-ant-..."
```

Notes by Siva Sai Charan / Gen AI Fundamentals - Section 3 Course: Generative AI Application Design and Development